

ТЕХНИЧЕСКИЙ ОБЗОР

Платформа «Опора» — Архитектура и технологический стек

Версия 2.1 | Июль 2026

1. Архитектурные принципы

Платформа построена на принципах **Clean Architecture + Domain-Driven Design (DDD)** с изолированными bounded contextами.

Принцип	Реализация
Микросервисная архитектура	20 сервисов, каждый со своей БД (Database per Service)
Clean Architecture	Слои: domain → application → infrastructure → interfaces
DDD	Bounded Context, Aggregates, Value Objects, Domain Events
CQRS	Разделение чтения и записи без Event Sourcing
Event-Driven	Apache Kafka для асинхронного обмена событиями
API-First	OpenAPI 3.0 спецификации до реализации

Ключевые ADR (Architecture Decision Records)

ADR	Решение	Статус
ADR-001	Миграция на микросервисы	✓ Исполнено
ADR-002	Стек Java 25 + Quarkus 3.36 + React 18	✓ Исполнено
ADR-003	Feature-Sliced Design для фронтенда	✓ Исполнено
ADR-004	CQRS без Event Sourcing	✓ Исполнено
ADR-005	API Gateway (Quarkus REST)	✓ Исполнено
ADR-006	Database per Service	✓ Исполнено
ADR-007	Mock External Services	✓ Исполнено

2. Технологический стек

Компонент	Версия	Назначение
Java	25	Основной язык бэкенда
Quarkus	3.36	Фреймворк для микросервисов (Kotlin DSL, Gradle 9.5.1)
React	18	Личный кабинет (SPA, TypeScript, Vite)
Apache Wicket	10.9.0	SSR для SEO-критичных страниц каталога
PostgreSQL	17	Реляционное хранилище (отдельная схема на сервис)
MongoDB	8.3	Документоориентированное хранилище
Redis	8.6.3	Кэширование, сессии
Apache Kafka	4.2.0	Интеграционная шина (события, синхронизация)
gRPC Java	1.81.0	Межсервисное взаимодействие
Apicurio Registry	3.2.x	Schema Registry (Avro)
Keycloak	24	Федеративная аутентификация (OIDC/OAuth 2.0)
Hyperledger Fabric	+ КriptoПро HLF	Блокчейн-прослеживаемость
ClickHouse	—	Аналитический контур, BI
Docker	—	Контейнеризация (multi-stage builds)
Kubernetes	—	Оркестрация (HPA, NetworkPolicy)

3. Декомпозиция сервисов (20 сервисов: 13 бизнес + 7 инфраструктурных)

3.1. Бизнес-модули (13)

Сервис	Назначение	Java- файлы	Архитектура
catalog-service	Витрина: каталог брендов, грейдирование, кешбэк	37	DDD
cooperation-service	Кооперация: AI-мэтчинг, артельные кластеры, тендеры	100	DDD

Сервис	Назначение	Java- файлы	Архитектура
creators-service	Креаторы: маркетплейс самозанятых	51	DDD
analytics-service	Аналитика: CRM/CDP, прогноз спроса	32	DDD
legal-service	Право: арбитраж, AI-юрист, патентный маркетплейс	37	DDD
finance-service	Финансы: краудфандинг 2.0, ЦФА, токены	15	Плоская
sandbox-service	Песочница: витрина за 5 минут, интеграция с «Мой налог»	32	DDD
chains-service	Цепочки: B2B-закупки, блокчейн-прослеживаемость	17	Плоская
personnel-service	Кадры: верификация компетенций, интеграция с вузами	16	Плоская
regions-service	Регионы: геотаргетинг, специфика моногородов	16	Плоская
export-service	Экспорт: выход на рынки ЕАЭС и БРИКС	27	DDD
loyalty-service	Лояльность: программа лояльности, баллы, кешбэк	35	DDD
payments-service	Платежи: платёжная инфраструктура, СБП, ЦФА	46	DDD

3.2. Инфраструктурные сервисы (7)

Сервис	Назначение	Java- файлы	Архитектура
auth-service	Keycloak + мок ЕСИА, аутентификация	21	DDD
gateway-service	API Gateway, rate limiting, circuit breaker	28	DDD
mock-service	8 моков внешних сервисов (ФНС, ОФД, СБП, ЦБ, ГИС)	25	—
lk-service	Личный кабинет (React SPA)	21	React SPA
bff-service	Backend-for-Frontend	28	DDD
admin-service	Администрирование платформы	17	Плоская
ai-service	ИИ-сервисы: рекомендации, аналитика, антифрод	22	Плоская

3.3. Shared-модули

Модуль	Статус	Содержимое
shared/domain	✗	Базовые DDD-типы (AggregateRoot, ValueObject, DomainEvent) — в разработке
shared/application	✗	Базовые application-сервисы — в разработке
shared/kafka-events	✓	DomainEvent.java, KafkaTopics.java

Итого: 626 Java-файлов, 20/20 сервисов с Docker. 14 сервисов на DDD-архитектуре, 6 — на плоской (в процессе рефакторинга).

4. Модель данных

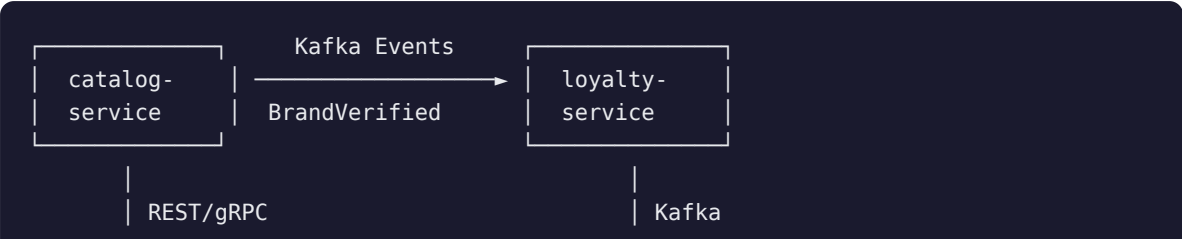
4.1. Принципы

- **Database per Service** — каждый сервис owns свою схему в PostgreSQL
- **Синхронизация через Kafka** — доменные события для кросс-сервисных данных
- **Eventual Consistency** — согласованность через идемпотентность и компенсационные транзакции

4.2. Ключевые сущности

Сервис	Агрегаты	Ключевые таблицы
catalog-service	Brand, Product, Grade, Cashback	brands, products, grades, cashback_rules
cooperation-service	Partnership, Cluster, Tender, Reputation	partnerships, clusters, tenders, reputation_scores
loyalty-service	Transaction, PointsBalance, AntiFraud	transactions, points_balances, antifraud_signals
chains-service	SupplyChain, Shipment, SmartContract	supply_chain_nodes, shipments, contracts
finance-service	Crowdfunding, Token, CreditScore	campaigns, tokens, credit_profiles

4.3. Интеграционные потоки





5. Архитектура безопасности

5.1. Класс защищённости K2

Требование	Реализация
Аутентификация	ЕСИА (OIDC/OAuth 2.0) через Keycloak
Авторизация	RBAC + ABAC, JWT-токены
Криптография	ГОСТ (КриптоПро CSP 5.0 R3)
Аудит	Логирование всех операций, immutable audit trail
Защита данных	Шифрование at rest (PostgreSQL TDE) + in transit (TLS 1.3)
Сетевая сегментация	Kubernetes NetworkPolicy, zero-trust
Пентест	Обязательный, ежегодный

5.2. Интеграции с госсистемами

Система	Протокол	Назначение
ЕСИА	OAuth 2.0/OIDC	Аутентификация граждан и юрлиц
ГИСП	REST API	Верификация продукции
Честный знак	REST API	Проверка маркировки
ОФД	REST/SOAP	Подтверждение покупок
ФНС	REST/SOAP	Налоговые данные
Мой налог	API	Самозанятые
ЦБ РФ	—	Цифровой рубль, смарт-контракты

5.3. Модель угроз (STRIDE)

Угроза	Контрмера
Spoofing	ЕСИА + MFA + JWT validation
Tampering	ГОСТ-подпись, immutable audit log
Repudiation	Blockchain-фиксация критичных операций
Information Disclosure	Encryption at rest + in transit, RBAC
Denial of Service	Rate limiting (100 req/min), WAF, circuit breaker
Elevation of Privilege	Least privilege, NetworkPolicy, Trivy scanning

6. Стратегия развёртывания

6.1. Инфраструктура

Компонент	Решение
Контейнеризация	Docker (multi-stage builds, JVM + native-image)
Оркестрация	Kubernetes (HPA, NetworkPolicy, StatefulSet)
CI/CD	GitHub Actions → Docker Registry → K8s
Мониторинг	Prometheus + Grafana + OpenTelemetry
Логирование	ELK Stack (Elasticsearch, Logstash, Kibana)
Трейсинг	Jaeger / OpenTelemetry

6.2. Стратегия развёртывания

Этап	Среда	Конфигурация
Разработка	Local (Docker Compose)	Все сервисы, моки внешних систем
Тестирование	minikube / K8s dev	Интеграционные тесты, BDD
Стейджинг	K8s staging	Production-like, нагрузочное тестирование
Продакшн	K8s production (ЦОД РФ)	Blue-Green деплой, SLA ≥99.5%

6.3. Нефункциональные требования

Параметр	Требование
Доступность	≥99.5% (пилот), ≥99.9% (продакшн)
Время ответа API	≤200ms (p95)
Пропускная способность	≥10 000 RPS (пиковая)
RTO (Recovery Time Objective)	≤4 часа
RPO (Recovery Point Objective)	≤1 час
Хранение данных	7 лет (требования ФСТЭК)

7. Тестовая стратегия

Уровень	Инструменты	Покрытие
Unit-тесты	JUnit 5 + Mockito	≥80% строк кода
Интеграционные	REST Assured + Testcontainers	Критичные пути
BDD	Cucumber	Пользовательские сценарии
Нагрузочные	Gatling	SLA-валидация
E2E	Playwright	UI-сценарии
Безопасность	Trivy, OWASP ZAP, FindSecBugs	Все сервисы

8. Статус реализации (июль 2026)

Категория	Статус
Java-файлы	626 (20 сервисов)
Docker	20/20 сервисов
DDD-архитектура	14/20 сервисов (6 — плоская, рефакторинг в процессе)
MVP-спецификации	75 документов
OpenAPI-контракты	10 спецификаций
C4-диаграммы	4 уровня (Context, Container, Component, Deployment)

Документ подготовлен на основе архитектурного проекта версии 2.1 (DOC-21) и аудита от 17.07.2026.

АО «НП «Опора» • Москва, 2026 • opora.ru • Строго конфиденциально